

Data Storage Design

DESIGN

TASK CHECKLIST

- ☒ Select Design Strategy.
- ☒ Design Architecture.
- ☒ Select Hardware and Software.
- ☒ Develop Use Scenarios.
- ☒ Design Interface Structure.
- ☒ Develop Interface Standards.
- ☒ Design Interface Prototype.
- ☒ Evaluate User Interface.
- ☒ Design User Interface.
- ☒ Develop Physical Data Flow Diagrams.
- ☒ Develop Program Structure Charts.
- ☒ Develop Program Specifications.
- ☐ Select Data Storage Format.
- ☐ Develop Physical Entity Relationship Diagram.
- ☐ Denormalize Entity Relationship Diagram.
- ☐ Performance Tune Data Storage.
- ☐ Size Data Storage.

Data Storage Formats

There are **two basic types** of data storage:

- Files
- Databases

Files

- Structured **to optimize a particular transaction.**
 - Usually in CSV or tab-delimited format.
 - Older systems kept data in their own format. (proprietary format)
 - Usually organized as a linked list.
- Types of files:
 - **Master files:** store core information
 - **Look-up files:** typically used for validation
 - **Transaction files:** holds transactions as they happen, used to update master data later
 - **Audit files:** keep before, after images of data
 - **History files:** holds past transactions, can be offline and accessed as-needed

Databases

- **A collection of files**, created and managed by a DBMS (Database Management System)
- **Many different types** of databases:
 - Relational Databases
 - Multidimensional Databases
 - NoSQL Databases
 - Object Databases
 - ...

Relational Databases

- Most popular for application development
- Collections of tables
 - Each table has a **primary key**
 - Tables are related to each other through **foreign keys**
- The DBMS guarantees **referential integrity**.
- **SQL** is used to create, read, update and delete data and structures (CRUD)

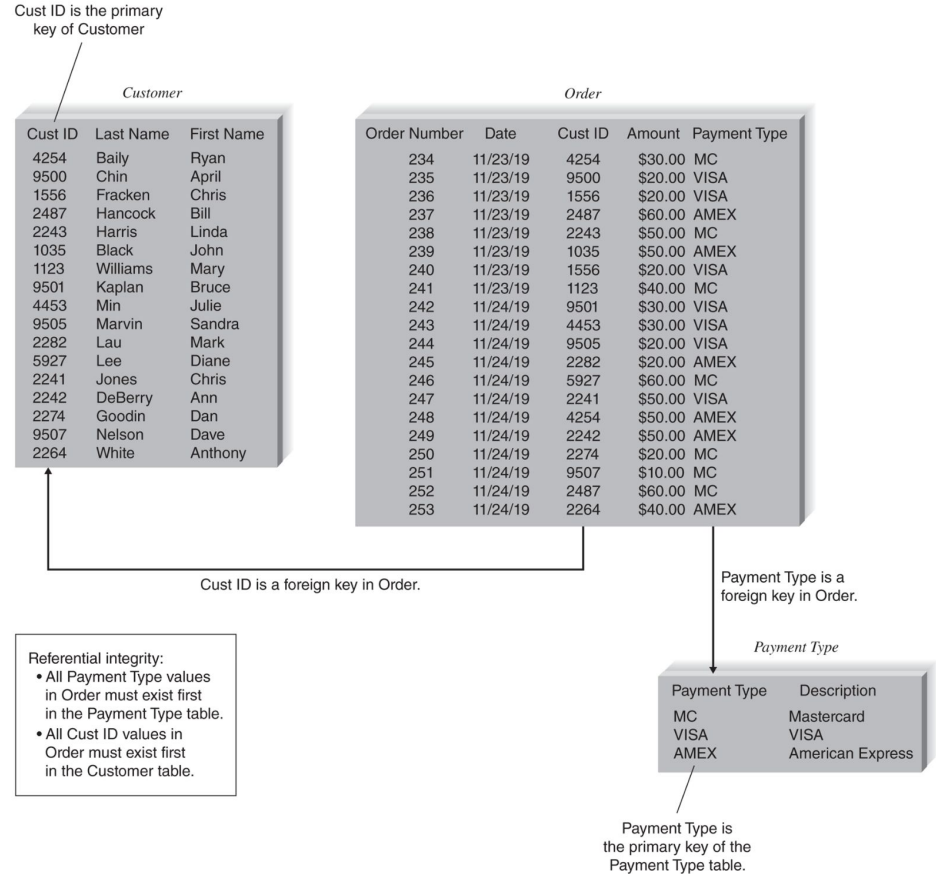
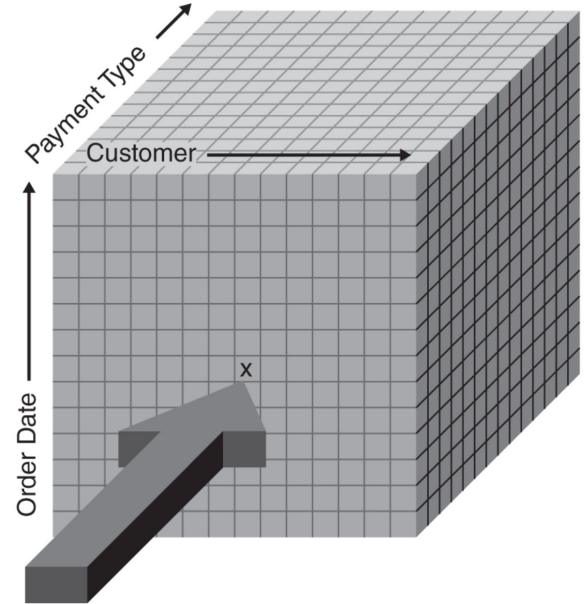


FIGURE 10-3 Relational database.

Multidimensional Databases

- Used extensively in **data warehousing** to support business intelligence systems.
- Data are organized as multidimensional **data cubes** so **facts** can be explored or **aggregated** on **various dimensions** quickly.
- Relational databases can be used to create and query these data cubes. (**star schema**)



Last quarter, how many customers placed more than one order, using an American Express card?

FIGURE 10-4 Multidimensional database.

NoSQL Databases

A wide variety of databases that do not use relational model and SQL:

- Document-oriented databases
 - A collection of **documents of varying structures**
- Wide column stores
 - Records can be extended to hold very **large numbers of dynamic columns**
- Graph databases
 - A collection of **nodes and edges**

Each storage format has its **strength & weaknesses**

	Files	Legacy databases 	Relational databases	Object databases	Multidimensional databases	NoSQL databases
Major strengths	Files can be designed for fast performance; good for short-term data storage.	Very mature products	Leader in the database market; can handle fast updating and querying needs	Able to handle complex data	Configured to answer business intelligence questions quickly	Designed for huge, varied data sets
Major weaknesses	Redundant data; data must be updated using programs.	<i>Not able to store data as efficiently; limited future</i>	Cannot handle complex data	Limited market acceptance; skills are hard to find.	Highly specialized use; skills are hard to find	New in the market, highly specialized use; skills are hard to find
Data types supported	Simple	<i>Not recommended for new systems</i>	Simple	Complex (e.g., video, audio, images)	Aggregated	Mixed data sets with structured and unstructured components
Application system types supported	Transaction processing	<i>Not recommended for new systems</i>	Transaction processing and decision making	Transaction processing	Business intelligence	Business intelligence; finding patterns and relationships in mixed data
Existing data formats	Organization dependent	Organization dependent	Organization dependent	Organization dependent	Organization dependent	Organization dependent
Future needs	Limited future prospects	Poor future prospects	Good future prospects	Uncertain future prospects	Uncertain future prospects	New, uncertain future prospects

FIGURE 10-5 Comparison of data storage formats.

Multiple storage formats might be used in a single project.

Selecting a storage format depends on:

- Data types
 - **Simple data**: text, dates, numbers
 - **Complex data**: video, images, audio
 - **Aggregated data**
 - **Big data**: huge *volume*, rapid *velocity*, great *variety*
- Type of application system
 - **Transaction processing systems**
 - **Business Intelligence (BI) systems**
- Existing storage formats
- Future needs

❏ Reading: Applying the Concepts at DronTeq

Moving from logical to physical data models

Step	Explanation
1. Change entities to tables or files.	Beginning with the logical ERD, change the entities to tables or files and update the metadata.
2. Change attributes to fields.	Convert the attributes to fields and update the metadata.
3. Add primary keys.	Assign primary keys to all entities.
4. Add foreign keys.	Add foreign keys to represent the relationships among entities.
5. Add system-related components.	Add system-related tables and fields.

FIGURE 10-7 Steps to moving from logical to physical entity relationship diagram.

Entities → Tables

- Rows & Columns
- Each row represents a single **entity instance**
- Each column represents an **attribute**
- Each intersection of a row and column is a single data value
- All values in a column have the same **data format**
- Each column has a range of values known as the **attribute domain**
- The order of rows and columns do not matter
- Each table must have an attribute or combination of attributes that uniquely identifies each row → **Primary Key**

Primary Key

- an identifier composed of **one or more attributes** that **uniquely identifies a row.**
- A table often has more than one **candidate** for primary key

1:M Relationship

- implemented by putting the PK of the “1” side in the table of the “Many” side as a **foreign key**
- **foreign key:** the primary key of one table placed in another table to create a common attribute

FIGURE 3.17 THE 1:M RELATIONSHIP BETWEEN PAINTER AND PAINTING



1:M Relationship

FIGURE 3.18 THE IMPLEMENTED 1:M RELATIONSHIP BETWEEN PAINTER AND PAINTING

Table name: PAINTER

Primary key: PAINTER_NUM

Foreign key: none

Database name: Ch03_Museum

PAINTER_NUM	PAINTER_LNAME	PAINTER_FNAME	PAINTER_INITIAL
123	Ross	Georgette	P
126	Ittero	Julio	G

Table name: PAINTING

Primary key: PAINTING_NUM

Foreign key: PAINTER_NUM

PAINTING_NUM	PAINTING_TITLE	PAINTER_NUM
1338	Dawn Thunder	123
1339	Vanilla Roses To Nowhere	123
1340	Tired Flounders	126
1341	Hasty Exit	123
1342	Plastic Paradise	126

FIGURE 3.19 THE 1:M RELATIONSHIP BETWEEN COURSE AND CLASS



1:M Relationship

FIGURE 3.20 THE IMPLEMENTED 1:M RELATIONSHIP BETWEEN COURSE AND CLASS

Table name: COURSE

Primary key: CRS_CODE

Foreign key: none

Database name: Ch03_TinyCollege

CRS_CODE	DEPT_CODE	CRS_DESCRIPTION	CRS_CREDIT
ACCT-211	ACCT	Accounting I	3
ACCT-212	ACCT	Accounting II	3
CIS-220	CIS	Intro. to Microcomputing	3
CIS-420	CIS	Database Design and Implementation	4
QM-261	CIS	Intro. to Statistics	3
QM-362	CIS	Statistical Applications	4

Table name: CLASS

Primary key: CLASS_CODE

Foreign key: CRS_CODE

CLASS_CODE	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10012	ACCT-211	1	M/W/F 8:00-8:50 a.m.	BUS311	105
10013	ACCT-211	2	M/W/F 9:00-9:50 a.m.	BUS200	105
10014	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10015	ACCT-212	1	M/W/F 10:00-10:50 a.m.	BUS311	301
10016	ACCT-212	2	Th 6:00-8:40 p.m.	BUS252	301
10017	CIS-220	1	M/W/F 9:00-9:50 a.m.	KLR209	228
10018	CIS-220	2	M/W/F 9:00-9:50 a.m.	KLR211	114
10019	CIS-220	3	M/W/F 10:00-10:50 a.m.	KLR209	228
10020	CIS-420	1	vV 6:00-8:40 p.m.	KLR209	162
10021	QM-261	1	M/W/F 8:00-8:50 a.m.	KLR200	114
10022	QM-261	2	TTh 1:00-2:15 p.m.	KLR200	114
10023	QM-362	1	M/W/F 11:00-11:50 a.m.	KLR200	162
10024	QM-362	2	TTh 2:30-3:45 p.m.	KLR200	162

Referential Integrity

- every **reference to an entity instance by another entity instance** is valid.
- in other words, making sure that **what we are linking to actually exists**.
- the FK should be **either null or a valid value**

Null Values

- nulls should be **avoided as much as possible**
- abundance of nulls is **a sign of poor design**

FIGURE 3.22 THE IMPLEMENTED 1:1 RELATIONSHIP BETWEEN PROFESSOR AND DEPARTMENT

Table name: PROFESSOR

Primary key: EMP_NUM

Foreign key: DEPT_CODE

Database name: Ch03_TinyCollege

EMP_NUM	DEPT_CODE	PROF_OFFICE	PROF_EXTENSION	PROF_HIGH_DEGREE
103	HIST	DRE 156	6783	Ph.D.
104	ENG	DRE 102	5561	MA
105	ACCT	KLR 229D	8665	Ph.D.
106	MKT/MGT	KLR 126	3899	Ph.D.
110	BIOL	AAK 160	3412	Ph.D.
114	ACCT	KLR 211	4436	Ph.D.
155	MATH	AAK 201	4440	Ph.D.
160	ENG	DRE 102	2248	Ph.D.
162	CIS	KLR 203E	2359	Ph.D.
191	MKT/MGT	KLR 409B	4016	DBA
195	PSYCH	AAK 297	3550	Ph.D.
209	CIS	KLR 333	3421	Ph.D.
228	CIS	KLR 300	3000	Ph.D.
297	MATH	AAK 194	1145	Ph.D.
299	ECON/FIN	KLR 284	2851	Ph.D.
301	ACCT	KLR 244	4683	Ph.D.
335	ENG	DRE 208	2000	Ph.D.
342	SOC	BBG 208	5514	Ph.D.
387	BIOL	AAK 230	8665	Ph.D.
401	HIST	DRE 156	6783	MA
425	ECON/FIN	KLR 284	2851	MBA
435	ART	BBG 185	2278	Ph.D.



The 1:M DEPARTMENT employs PROFESSOR relationship is implemented through the placement of the DEPT_CODE foreign key in the PROFESSOR table.



The 1:1 PROFESSOR chairs DEPARTMENT relationship is implemented through the placement of the EMP_NUM foreign key in the DEPARTMENT table.

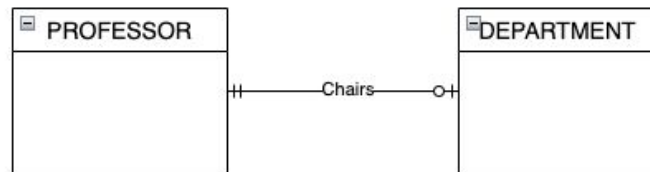
Table name: DEPARTMENT

Primary key: DEPT_CODE

Foreign key: EMP_NUM

DEPT_CODE	DEPT_NAME	SCHOOL_CODE	EMP_NUM	DEPT_ADDRESS	DEPT_EXTENSION
ACCT	Accounting	BUS	114	KLR 211, Box 52	3119
ART	Fine Arts	A&SCI	435	BBG 185, Box 128	2278
BIOL	Biology	A&SCI	387	AAK 230, Box 415	4117
CIS	Computer Info. Systems	BUS	209	KLR 333, Box 56	3245
ECON/FIN	Economics/Finance	BUS	299	KLR 284, Box 63	3126
ENG	English	A&SCI	160	DRE 102, Box 223	1004
HIST	History	A&SCI	103	DRE 156, Box 284	1867
MATH	Mathematics	A&SCI	297	AAK 194, Box 422	4234
MKT/MGT	Marketing/Management	BUS	106	KLR 126, Box 55	3342
PSYCH	Psychology	A&SCI	195	AAK 297, Box 438	4110
SOC	Sociology	A&SCI	342	BBG 208, Box 132	2008

1:1 Relationship

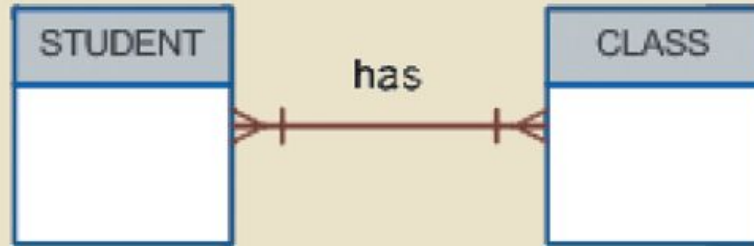


- We could potentially put the PK of DEPARTMENT into the PROFESSOR table. But that would be less ideal as not all professors are department heads (leads to lots of nulls). But each department has a head.

M:N Relationship

- **Relationships** are implemented through common attributes.
- Which common attributes should we define among the two tables?

FIGURE 3.23 THE ERM'S M:N RELATIONSHIP BETWEEN STUDENT AND CLASS



M:N Relationship

- create an intersection entity
 - also called composite entity, **bridge entity** or associative entity
- Include the PK of the two related entities in the intersection entity as foreign keys.
- **PK of the intersection entity** then can be a composite key (combination of the two FKs) or the designer can introduce a new PK

FIGURE 3.25 CONVERTING THE M:N RELATIONSHIP INTO TWO 1:M RELATIONSHIPS

Table name: STUDENT
Primary key: STU_NUM
Foreign key: none

STU_NUM	STU_LNAME
321452	Bowser
324257	Smithson

Database name: Ch03_CollegeTry2

Table name: ENROLL
Primary key: CLASS_CODE + STU_NUM
Foreign key: CLASS_CODE, STU_NUM

CLASS_CODE	STU_NUM	ENROLL_GRADE
10014	321452	C
10014	324257	B
10018	321452	A
10018	324257	B
10021	321452	C
10021	324257	C

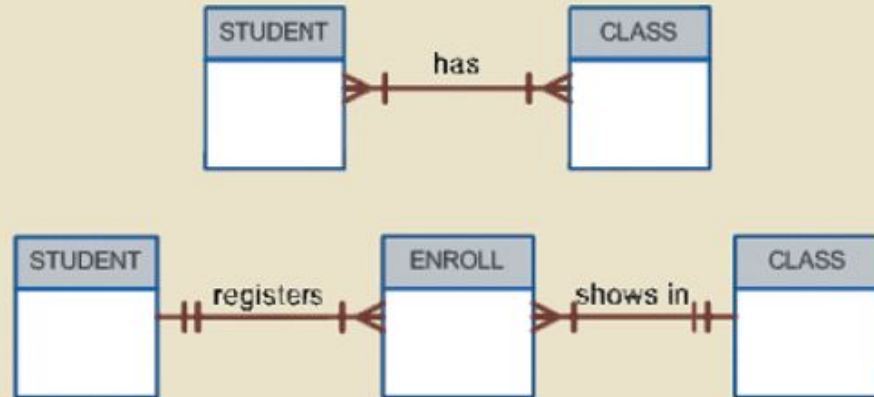
Table name: CLASS
Primary key: CLASS_CODE
Foreign key: CRS_CODE

CLASS_CODE	CRS_CODE	CLASS_SECTION	CLASS_TIME	CLASS_ROOM	PROF_NUM
10014	ACCT-211	3	TTh 2:30-3:45 p.m.	BUS252	342
10018	CIS-220	2	M/W/F 9:00-9:50 a.m.	KLR211	114
10021	GM-261	1	M/W/F 8:00-8:50 a.m.	KLR200	114

the linking table can contain
any relevant attribute

controlled redundancy in the ENROLL table **will not**
lead to anomalies if the referential integrity is enforced

FIGURE 3.26 CHANGING THE M:N RELATIONSHIPS TO TWO 1:M RELATIONSHIPS



Optimizing Data Storage

Two things to optimize:

1. Storage Efficiency
2. Speed of Access

These two goals often **conflict**.

Storage Efficiency

Redundant data and null values should be minimized as much as possible.
In addition to **wasting space**, redundancy and null values threaten **data integrity**.

- Redundancy increases the likelihood of **anomalies**.
- Null values are difficult, often impossible, to interpret.

CUSTOMER ORDER													
Order Number													
Date													
Cust ID													
Last Name													
First Name													
State													
Amount													
Tax Rate													
Product 1													
Product Description 1													
Product 2													
Product Description 2													
Product 3													
Product Description 3													

Order Number	Date	Cust ID	Last Name	First Name	State	Amount	Tax Rate	Product	Product Desc	Product	Product Desc	Product	Product Desc
239	11/23/19	1135	Black	John	MD	\$50.00	0.05	555 Cheese Tray					
260	11/24/19	1135	Black	John	MD	\$40.00	0.05	444 Wine Gift Pack					
273	11/27/19	1135	Black	John	MD	\$20.00	0.05	222 Bottle Opener					
241	11/23/19	1123	Williams	Mary	CA	\$40.00	0.08	444 Wine Gift Pack					
262	11/24/19	1123	Williams	Mary	CA	\$20.00	0.08	222 Bottle Opener					
287	11/27/19	1123	Williams	Mary	CA	\$20.00	0.08	222 Bottle Opener					
290	11/30/19	1123	Williams	Mary	CA	\$50.00	0.08	555 Cheese Tray					
234	11/23/19	2242	DeBerry	Ann	DC	\$50.00	0.065	555 Cheese Tray					
237	11/7/19	2242	DeBerry	Ann	DC	\$50.00	0.065	111 Wine Guide		444 Wine Gift Pack			
238	11/10/19	2242	DeBerry	Ann	DC	\$40.00	0.065	444 Wine Gift Pack					
245	11/11/19	2242	DeBerry	Ann	DC	\$20.00	0.065	222 Bottle Opener					
250	11/18/19	2242	DeBerry	Ann	DC	\$20.00	0.065	222 Bottle Opener					
252	11/22/19	2242	DeBerry	Ann	DC	\$60.00	0.065	222 Bottle Opener		444 Wine Gift Pack			
253	11/23/19	2242	DeBerry	Ann	DC	\$60.00	0.065	222 Bottle Opener		444 Wine Gift Pack			
297	11/24/19	2242	DeBerry	Ann	DC	\$30.00	0.065	333 Jams & Jellies					
243	11/11/19	4254	Bailey	Ryan	MD	\$50.00	0.05	555 Cheese Tray					
246	11/18/19	4254	Bailey	Ryan	MD	\$30.00	0.05	333 Jams & Jellies					
248	11/22/19	4254	Bailey	Ryan	MD	\$60.00	0.05	222 Bottle Opener		333 Jams & Jellies		111 Wine Guide	
235	11/17/19	9500	Chin	April	KS	\$20.00	0.05	222 Bottle Opener					
242	11/23/19	9500	Chin	April	KS	\$30.00	0.05	333 Jams & Jellies					
244	11/24/19	9500	Chin	April	KS	\$20.00	0.05	222 Bottle Opener					
251	11/27/19	9500	Chin	April	KS	\$10.00	0.05	111 Wine Guide					

FIGURE 10-12 Optimizing data storage.

Normalization is the best way to optimize data storage for efficiency.

Optimizing Access Speed

- When we need to access data from multiple tables, we should first join the table together.
- Joins can take a long time, especially if tables are large or if many tables are involved.
- Several techniques to speed up access to data:
 - Denormalization
 - Clustering
 - Indexing
 - Estimating the size of data (for hardware planning)

Denormalization (adding back redundancy)

- Should be applied sparingly
- Ideal in situations in which information is **queried frequently** yet **updated rarely**.
- Denormalization can reduce joins and improve performance in:
 - Look up tables
 - 1:1 relationships
 - 1:N relationships
 - Star Schema Designs

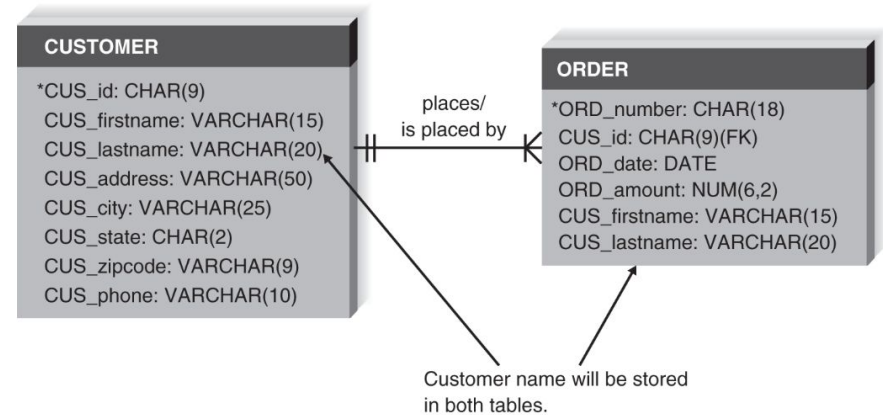


FIGURE 10-13 Denormalized data model.

❏ Reading: Reasons to denormalize

Indexing

- An index works **similar to a library catalogue**
- An index consists of an **index key** and a **set of pointers** to table rows
- This arrangement of (key, pointers) helps us access table rows more quickly.

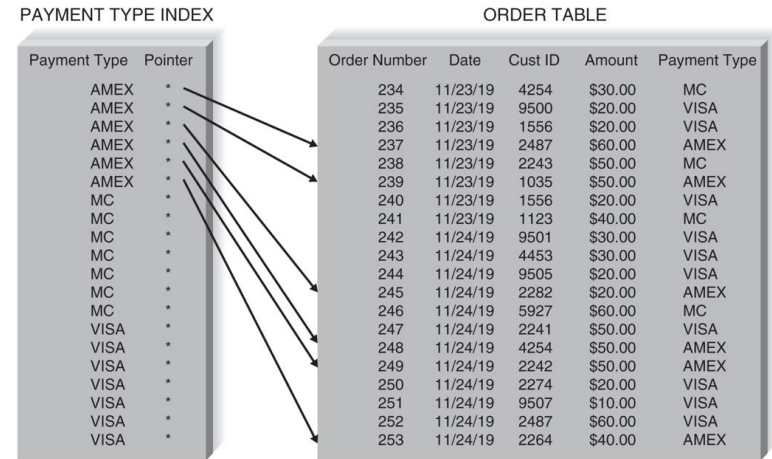


FIGURE 10-15 Payment type index.

Where to use indexes?

- Each index is associated only with one table
- A table can have many indexes

- Use indexes sparingly for transaction systems.
- Use many indexes to improve response times in business intelligence systems.
- For each table, create a unique index that is based on the primary key.
- For each table, create an index that is based on the foreign key to improve the performance of joins.
- Create an index for fields that are used frequently for grouping, sorting, or criteria.

FIGURE 10-16 Guidelines for creating indexes.

Indexes require **overhead**: they take up space and also need to be updated as records are inserted, deleted or changed:

- Use them **sparingly** on data that will be updated a lot.
- Use them **generously** on read-only data, such as in decision support systems.

❏ Reading: Estimating Storage Size